

Sesión 12. Modelos de secuencias – Redes neuronales recurrentes

- Modelos de secuencias.
- Redes neuronales recurrentes.
- Redes con memoria

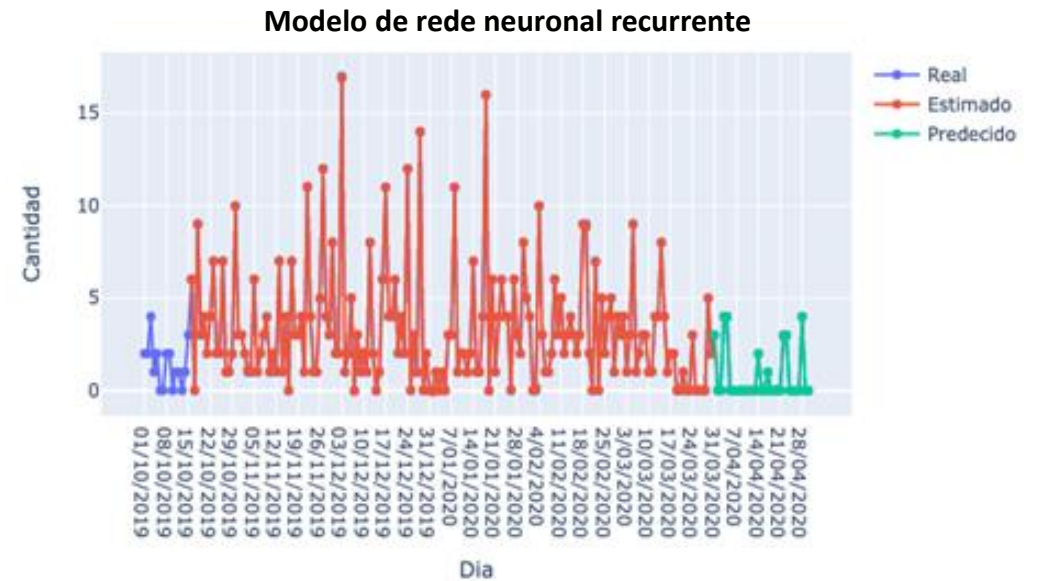
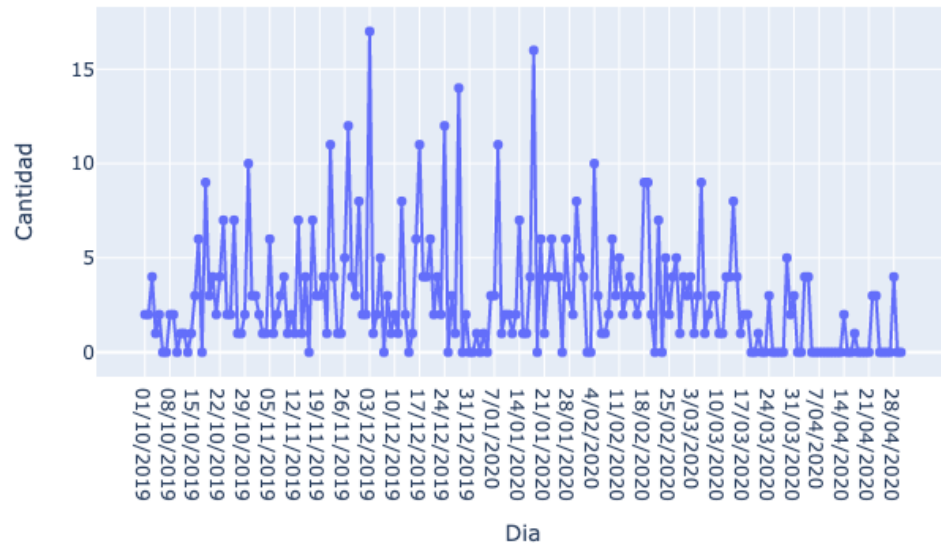
Modelos de secuencias

Algunas aplicaciones

- Gestión de pedidos para puntos de ventas con deep learning*.

Objetivo: automatizar un sistema de pedidos para los diferentes puntos de venta, con el fin de reducir el sobre stock.

Solución: modelo capaz de ajustarse al comportamiento de demanda de los productos.



Empresa comercializadora de productos naturales con presencia a nivel nacional. Cuenta con más de 1300 productos registrados en alrededor de 170 puntos de venta. Actualmente, todos los pedidos se realizan a una bodega central en Bogotá.

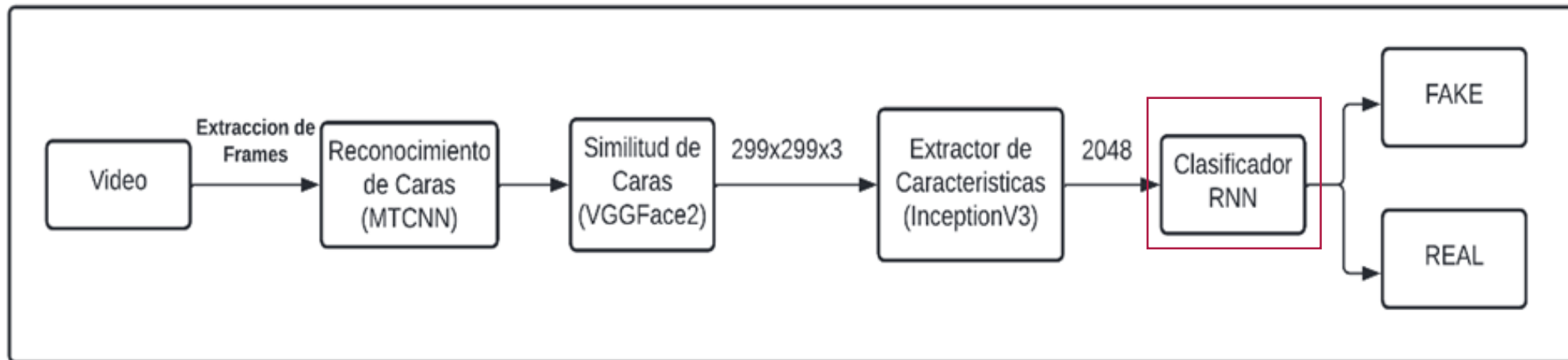
- Detección de videos multi-rostro manipulados por DeepFake con aprendizaje profundo.

Objetivo: desarrollo de un modelo que permita clasificar los rostros de un video como reales o alterados por DeepFake.

Solución: red neuronal convolucional (CNN), la cual se utiliza como extractor de características, y una red recurrente para la clasificación.



Técnica, basada en deep learning, para editar videos falsos de personas aparentemente reales.



El rostro es : REAL
Con una probabilidad de: 99.99%

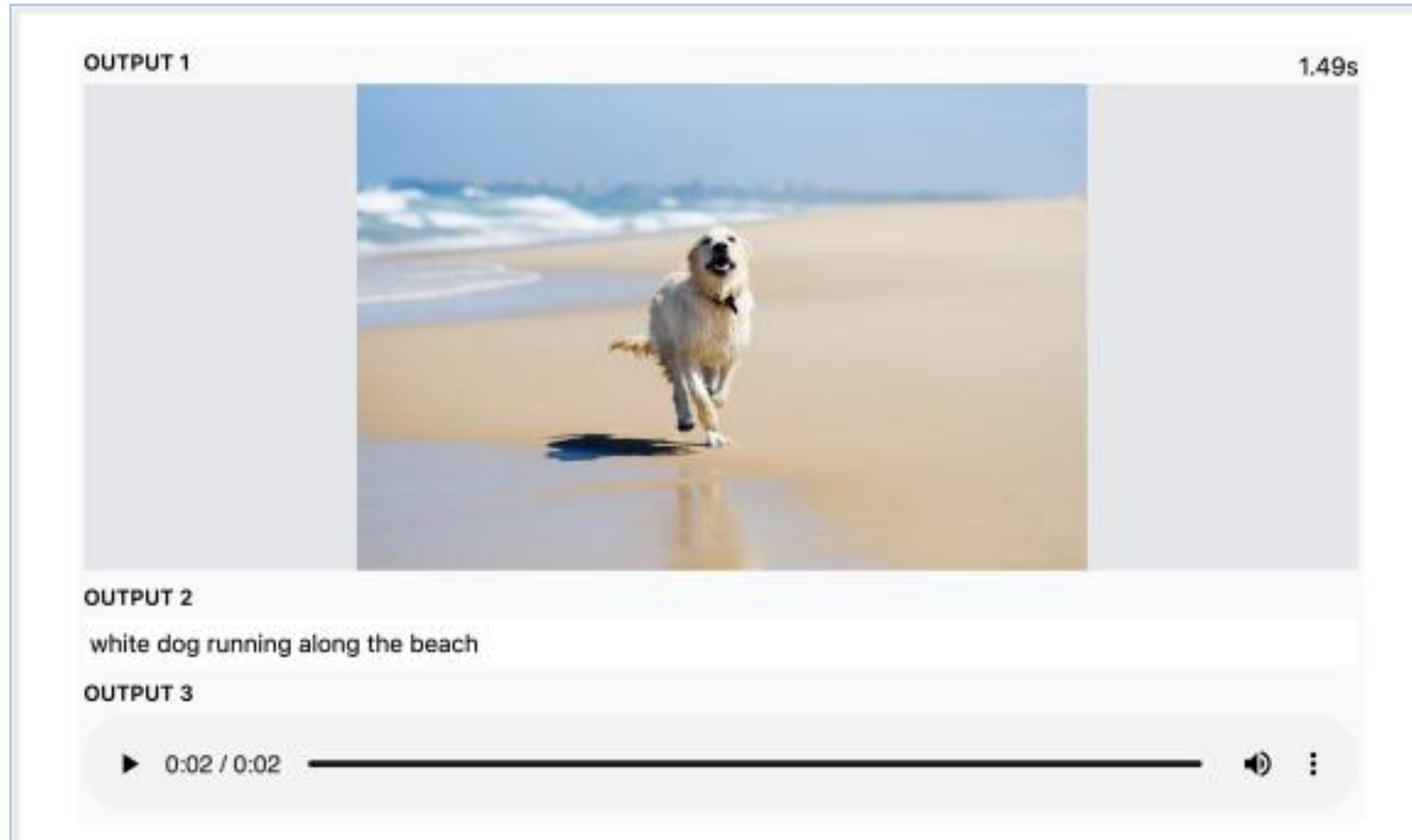


El rostro es : REAL
Con una probabilidad de: 99.97%

- **Descriptor de alrededores a partir de imágenes digitales para individuos con discapacidad visual.**

Objetivo: desarrollar una herramienta capaz de describir lo que ocurre en una imagen tomada por un usuario con dificultades de visión, con el fin de brindar información sobre lo que está ocurriendo en sus alrededores, que puede serle de utilidad para decidir su siguiente paso.

Solución: se construyó un modelo con técnicas de aprendizaje profundo que genera una descripción escrita a partir de una imagen. Este modelo se conecta con la API text to speech de Google con el fin de brindar información auditiva al usuario sobre lo que ocurre a su alrededor.



❏ ¿Qué es una secuencia?

- Para muchos problemas: datos multidimensionales de longitud fija con atributos independientes.
- Pero hay otros conjuntos de datos que exhiben dependencias temporales o secuenciales. Por ejemplo:

- ➡ Series de tiempo
- ➡ Textos, expresiones habladas,...
- ➡ Datos biológicos, como secuencias de aminoácidos.
- ➡ Lecturas de sensores
- ➡ Accesos a la Web
- ➡ Videos
- ➡ Datos climáticos.

Algunas aplicaciones:

- Pronóstico de series de tiempo.
- Generación de música.
- Descripción de actividades en videos.
- Elaboración de resúmenes de textos.
- Identificación de proteínas en secuencias biológicas.
- Identificación de entidades nombradas.
- Sistemas de preguntas – respuestas,

¿Qué es una secuencia?

Una **secuencia** es una lista ordenada de **TOKENS**, donde cada token representa una **unidad de información**.

¿Qué puede ser un token?

TEXTO (palabras)



Cada token es una palabra de una oración.

VIDEO (frames)



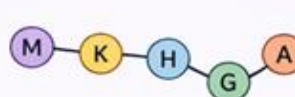
Cada token es un frame (imagen) del video.

SERIES DE TIEMPO (números)



Cada token es un valor numérico en el tiempo.

BIOLOGÍA (aminoácidos)



Cada token es un aminoácido en una secuencia de proteínas.

OTROS EJEMPLOS (sonido, eventos, clicks, etc.)



Cada token puede representar cualquier tipo de evento o medida.

EJEMPLO "El gato está en la casa"

El gato está en la casa
1 2 3 4 5 6

Frame 1 Frame 2 Frame 3 ...
 ...
1 2 3 ...

Temperatura diaria (°C)
21.4 22.1 20.8 19.7 21.6 ...
1 2 3 4 5 ...

Secuencia de aminoácidos
M K H G A L ...
1 2 3 4 5 6 ...

Nivel de audio (cada 0.1s)
0.03 0.05 0.02 0.07 0.04 ...
1 2 3 4 5 ...

¿QUÉ ES ENTONCES UNA SECUENCIA?

Token₁ → Token₂ → Token₃ ... Token_t
Paso 1 Paso 2 Paso 3 Paso t

Una secuencia es una colección **ordenada** de tokens:

- ✓ Los tokens llegan uno tras otro en el tiempo o en el espacio.
- ✓ El orden contiene información importante.

¿POR QUÉ IMPORTA EL ORDEN?

Mismo conjunto de tokens,
distinto orden, distinto significado.

✓ El gato muerde al perro 😊 Tiene sentido
✗ Perro al muerde gato el 😞 No tiene sentido



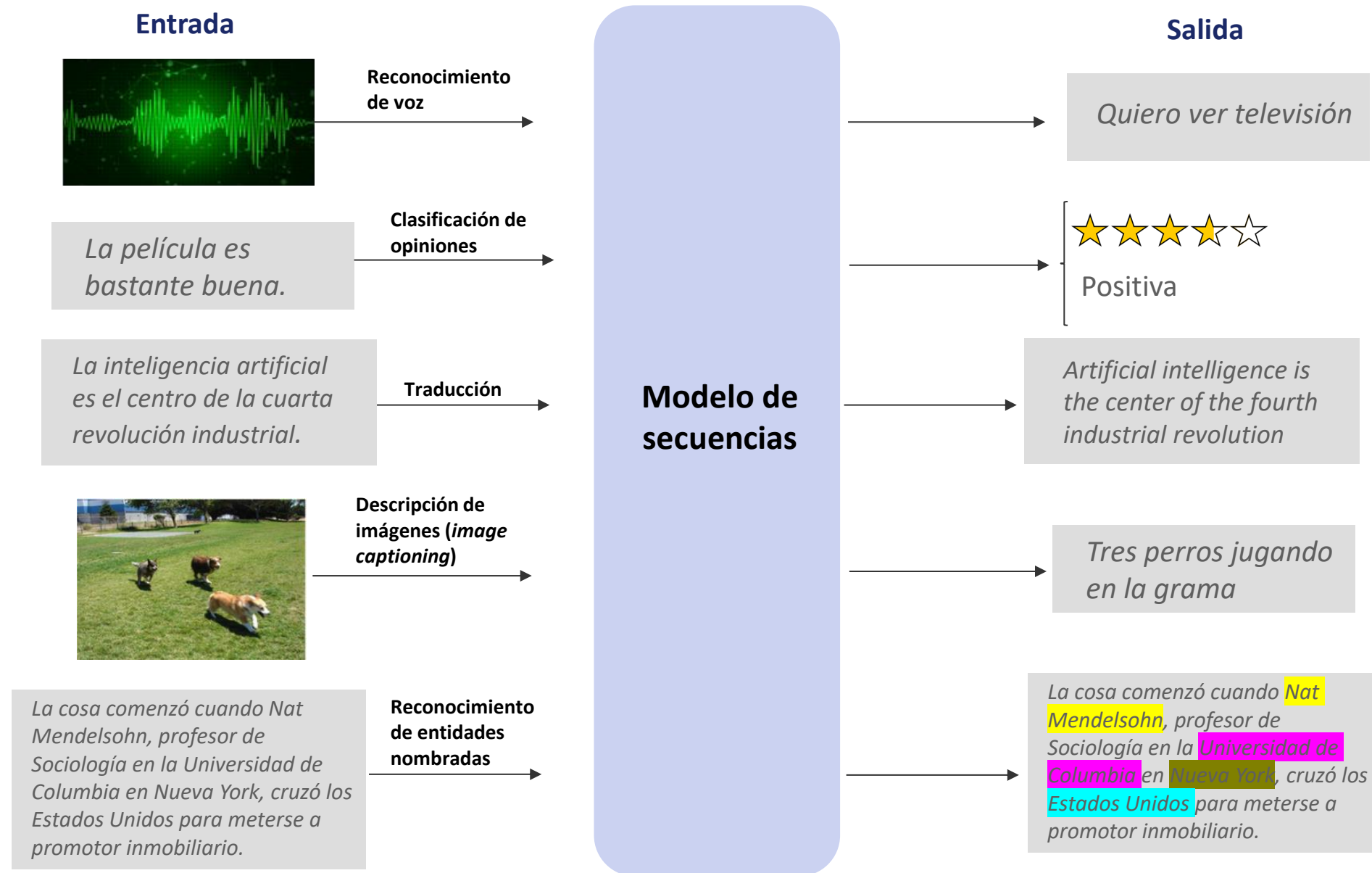
EN RESUMEN

Las secuencias están en todas partes: lenguaje, videos, series de tiempo, biología y más. Entender y modelar el **orden** es clave para resolver muchos problemas del mundo real.

Estas secuencias ¿Cómo procesarlas?

Modelos de secuencia (*sequence models*)

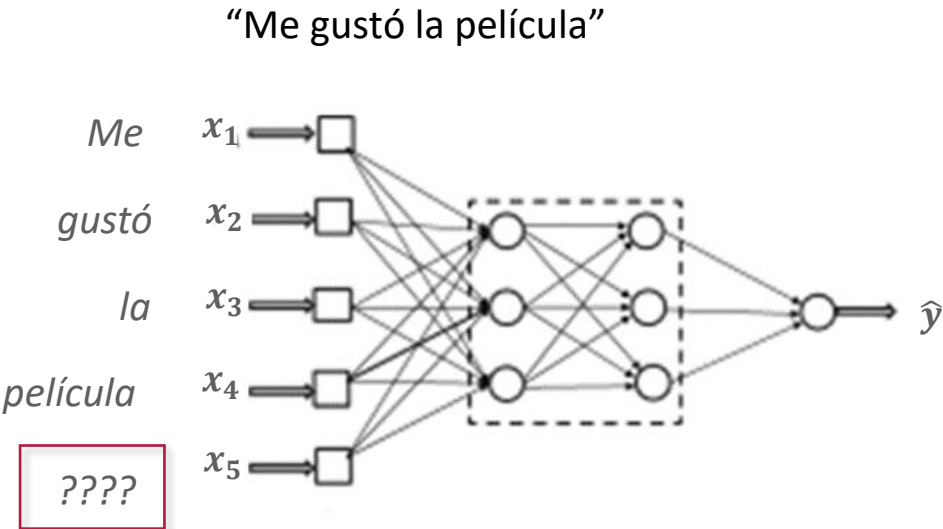
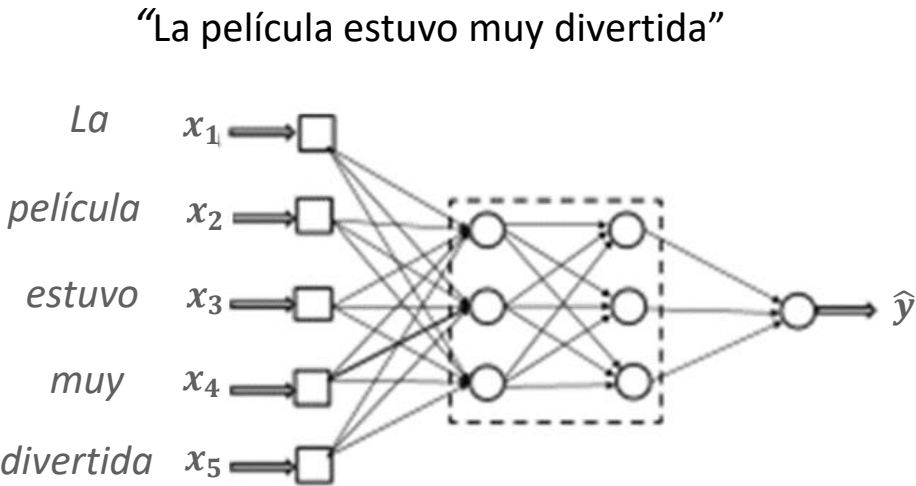
Ejemplos:



Ejemplo: análisis de sentimientos/opiniones

¿Qué pasa si se utiliza una red neuronal convencional?

Texto	Polaridad
La película estuvo bastante buena, sobre todo al final.	Positiva
Al principio captó mi atención y los actores son buenos, pero me quedé dormido.	Negativa
La película estuvo muy divertida	Positiva
Me gustó la película	Positiva



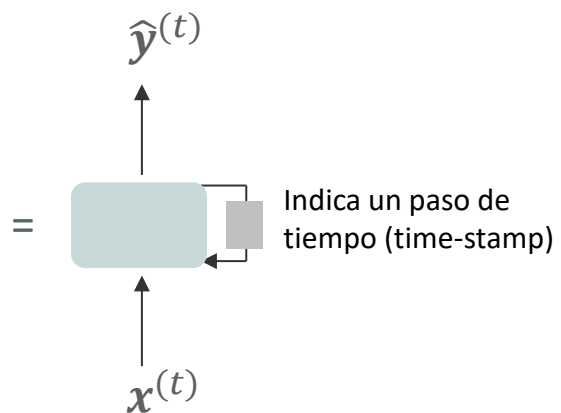
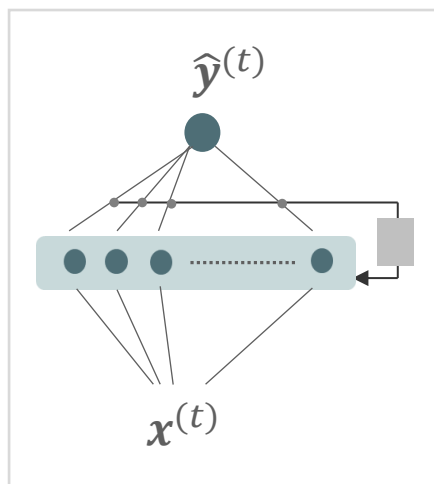
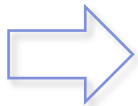
¿Qué se requiere?

- ☞ Habilidad para procesar entradas de longitudes diferentes.
- ☞ Habilidad para recibir y procesar la entrada en el mismo orden en que se presenta la secuencia.
- ☞ Tratamiento similar a entradas similares con base en la historia previa.

Red neuronal con un número fijo de parámetros, pero con la capacidad de procesar un número variable de entradas.

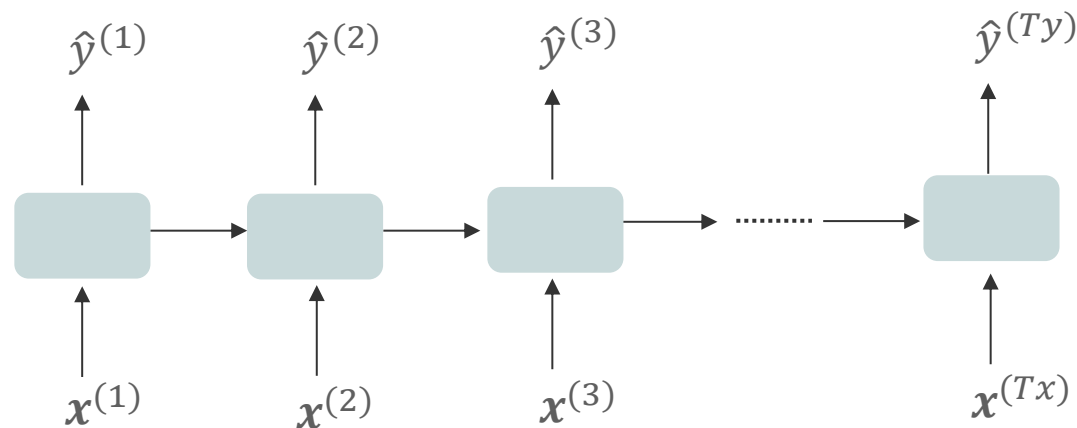


**Red neuronal
recurrente**



$T_x = \text{Longitud de la secuencia de entrada}$

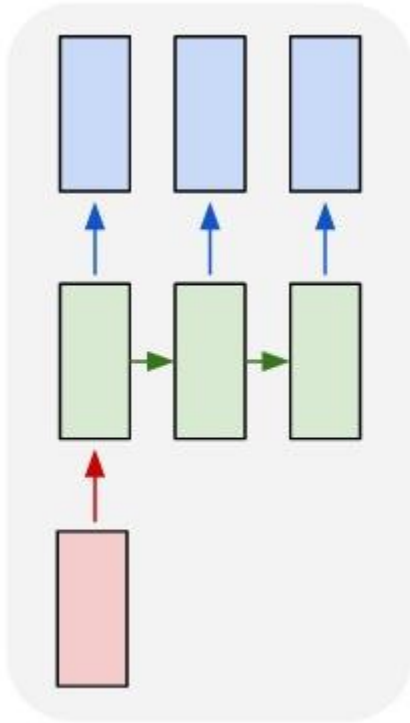
$T_y = \text{Longitud de la secuencia de salida}$



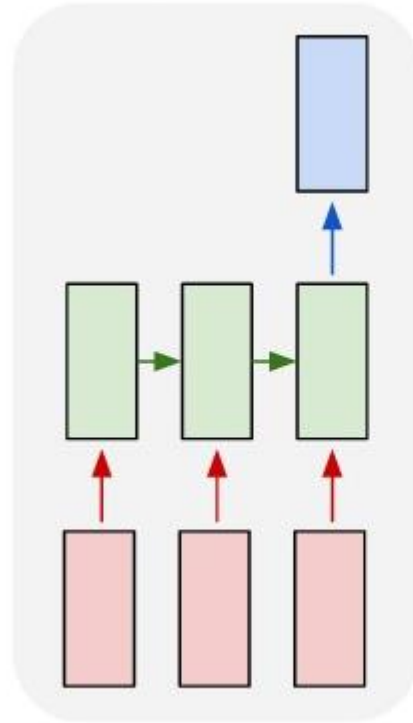
Representación “unrolled” de la red recurrente

❑ Modelos de secuencias con RNN:

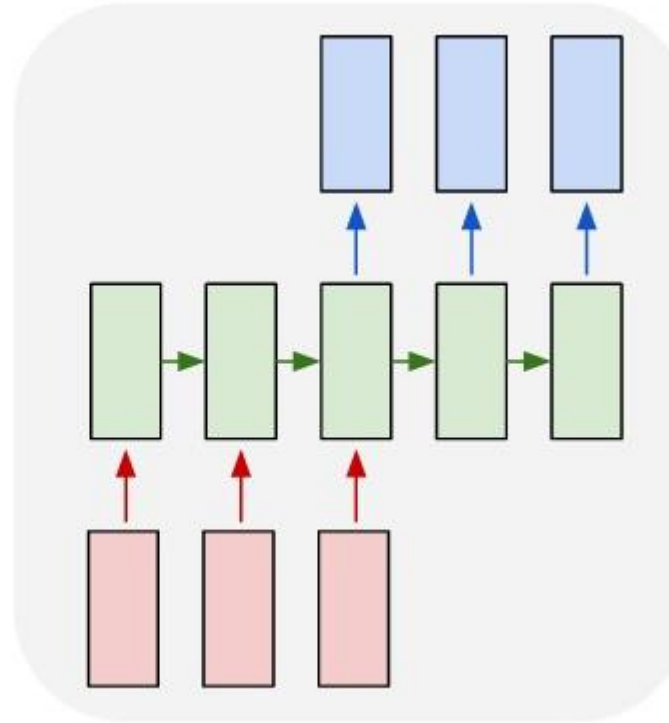
Uno a muchos



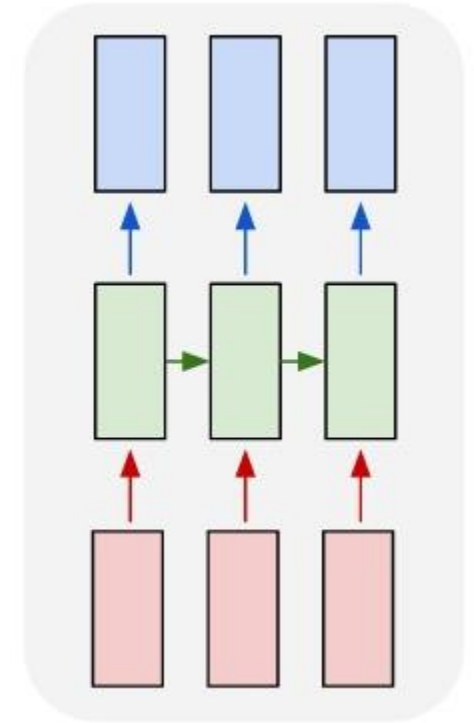
Muchos a uno



Muchos a muchos



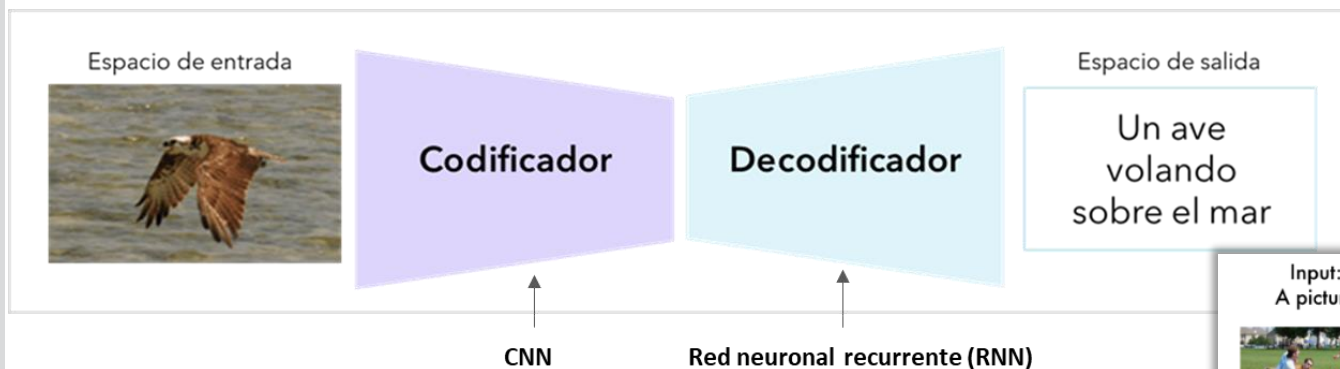
Muchos a muchos



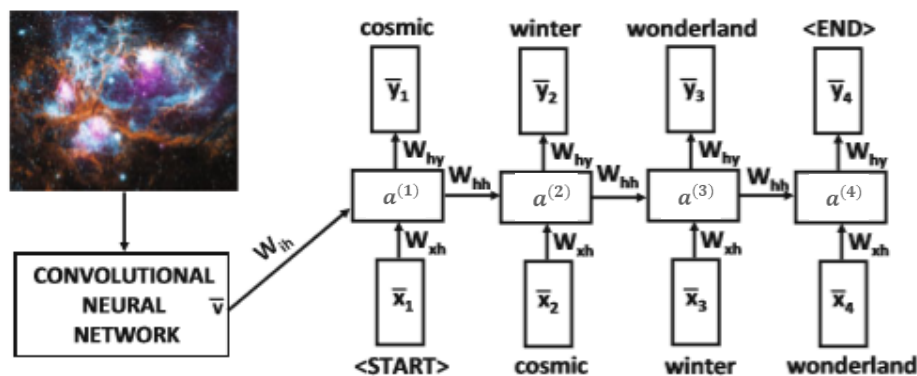
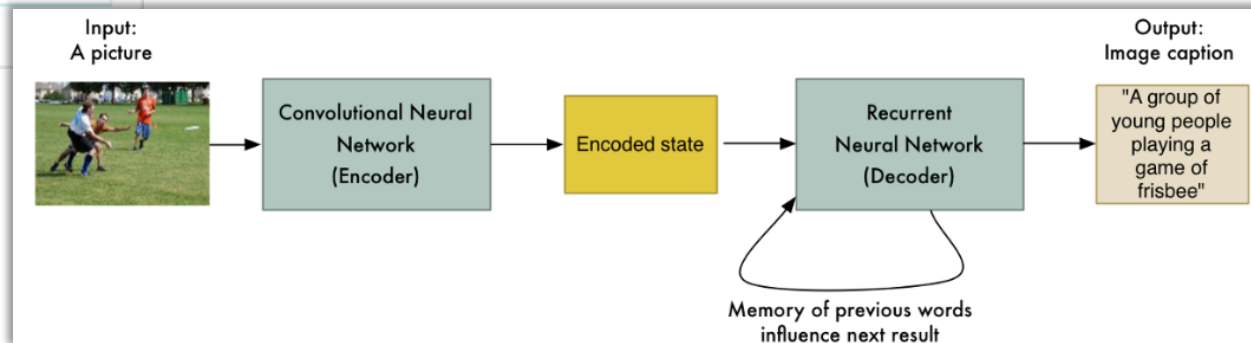
¿Para cuáles problemas se adecua cada uno?

■ Arquitectura uno a muchos

Ejemplo: descripción/titulación de imágenes .



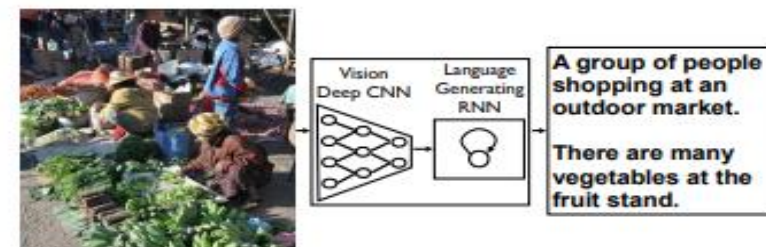
Show and Tell: A Neural Image Caption Generator (Google).
<http://static.googleusercontent.com/media/research.google.com/es//pubs/archive/43274.pdf>



$$a^{(1)} = f(W_{ih}v + W_{xh}x_1)$$

v = vector de salida de la CNN
 (representación de la imagen).

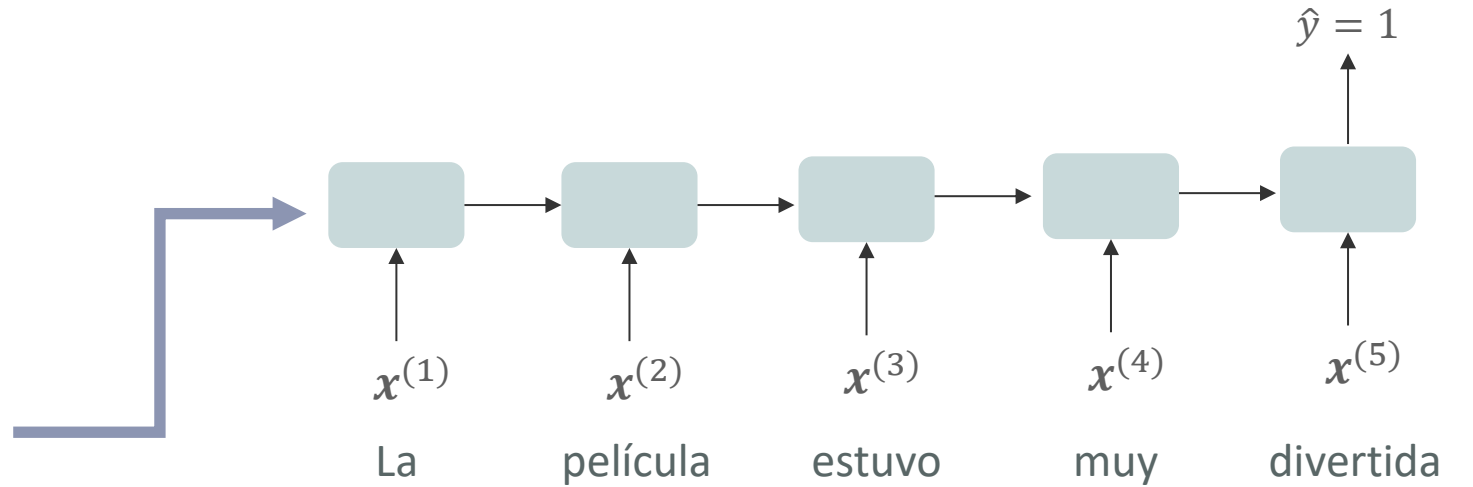
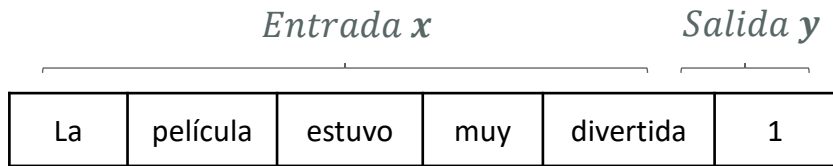
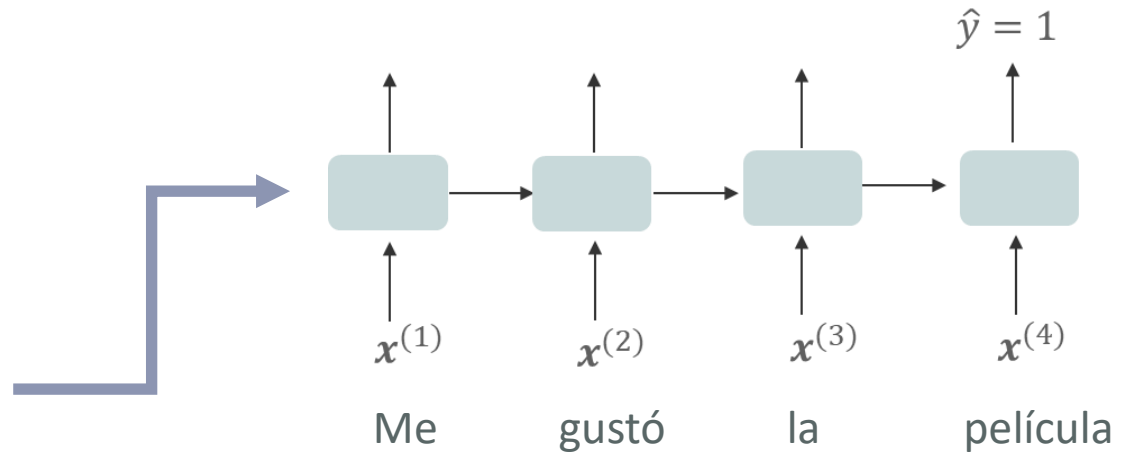
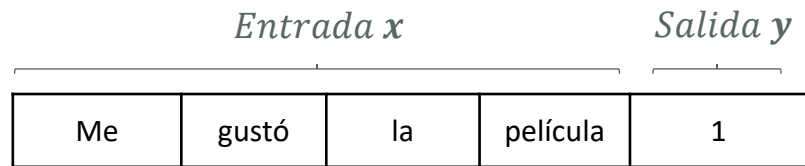
W_{ih} = procesa la representación
 de la imagen, v , para la capa
 oculta.



Charu C. Aggarwal. (2018). Neural Networks and Deep Learning.
 A Textbook. Springer.

- Arquitectura muchos a uno

Ejemplo: análisis de sentimientos/opiniones



- Arquitectura muchos a muchos ($Tx = Ty$)

Ejemplo: extracción de información y reconocimiento de entidades nombradas (named entity recognition)

- Extracción de información estructurada a partir del texto.
- Esta información pueden ser *entidades* de cierto tipo (por ejemplo, organizaciones, personas o nombres, localizaciones, ...), o *relaciones semánticas* (por ejemplo, *sintoma_de*, *autor_de*,...).

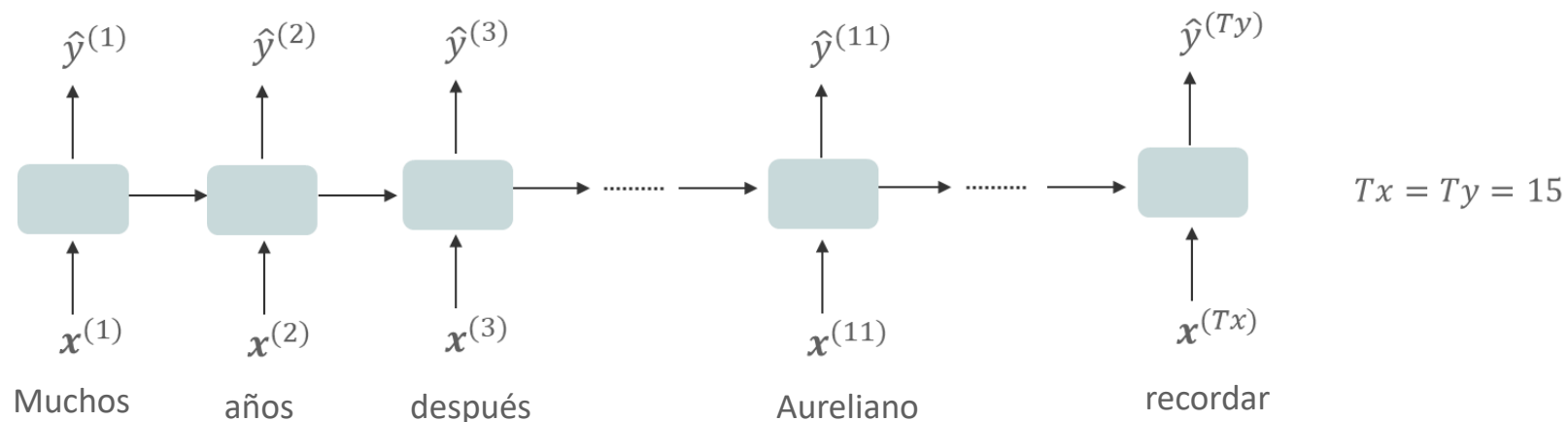


Entrada x :

Muchos	años	después	frente	al	pelotón	de	fusilamiento	el	coronel	Aureliano	Buendía	había	de	recordar
--------	------	---------	--------	----	---------	----	--------------	----	---------	-----------	---------	-------	----	----------

Salida y :

0	0	0	0	0	0	0	0	0	0	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



- Arquitectura muchos a muchos ($T_x \neq T_y$)

Ejemplo: traducción



Entrada x :

La	inteligencia	artificial	es	el	centro	de	la	cuarta	revolución	industrial
----	--------------	------------	----	----	--------	----	----	--------	------------	------------

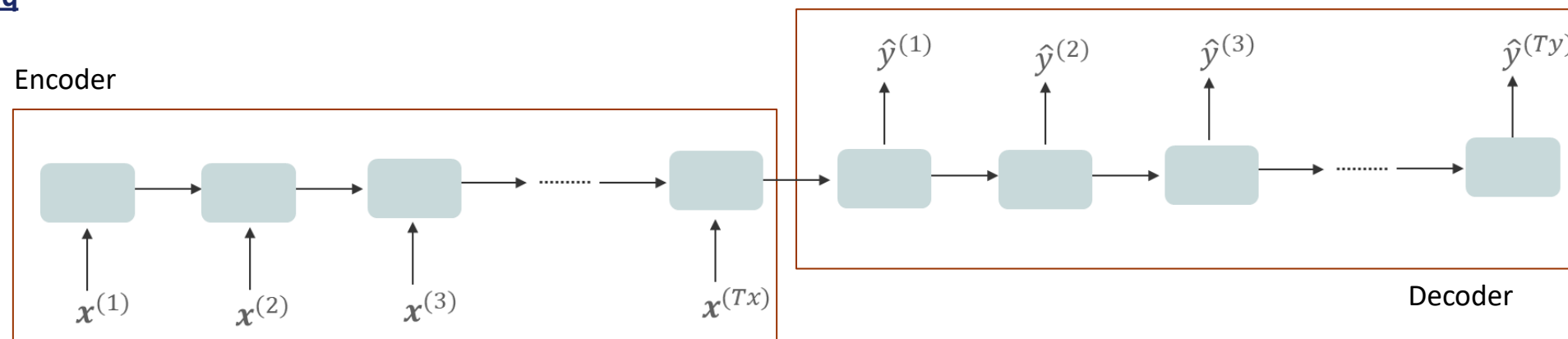
$$T_x = 11$$

Salida y :

Artificial	intelligence	is	the	center	of	the	fourth	industrial	revolution
------------	--------------	----	-----	--------	----	-----	--------	------------	------------

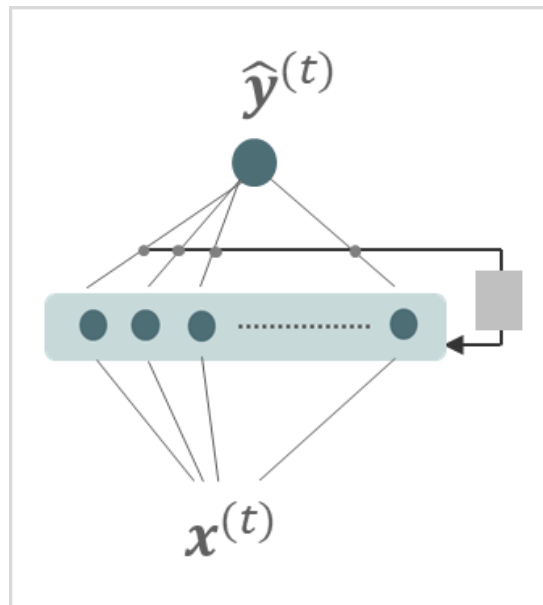
$$T_y = 10$$

Modelo seq2seq



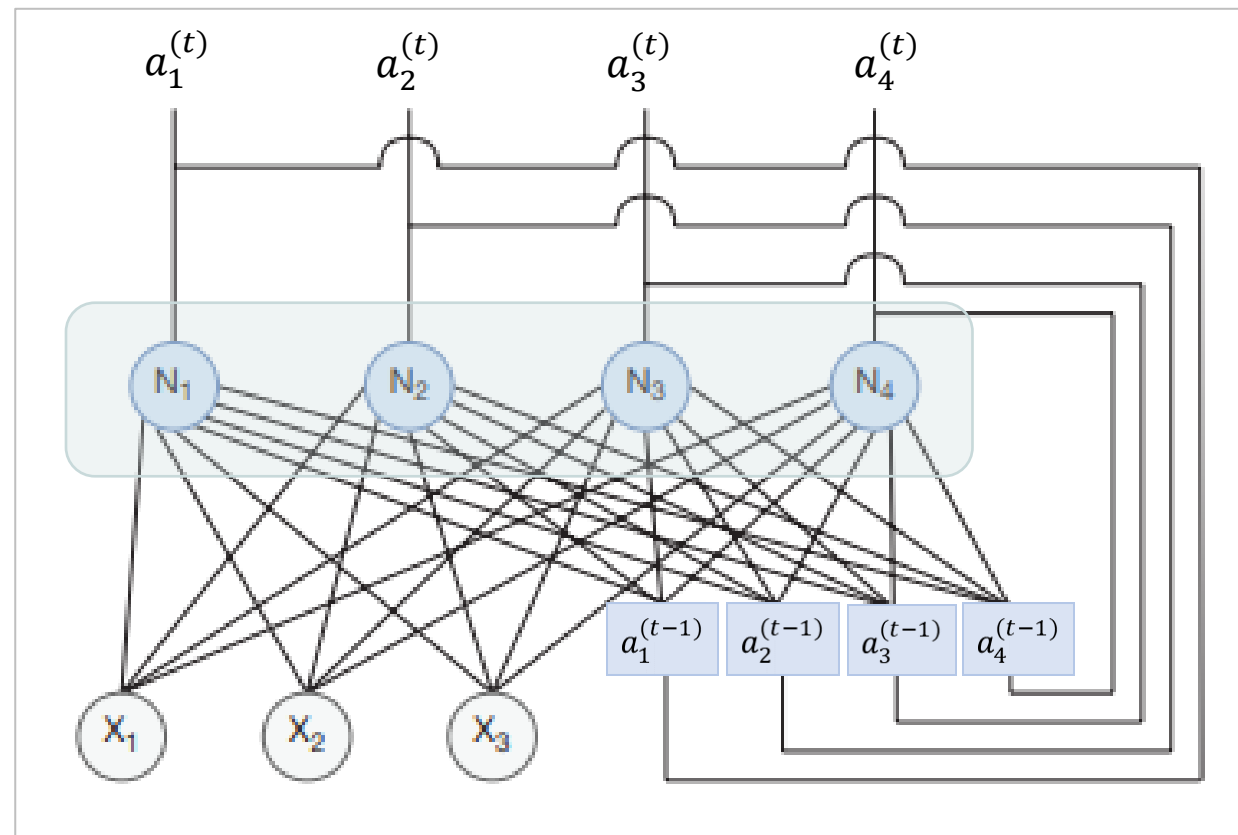
$$T_x \neq T_y$$

❑ Operación interna



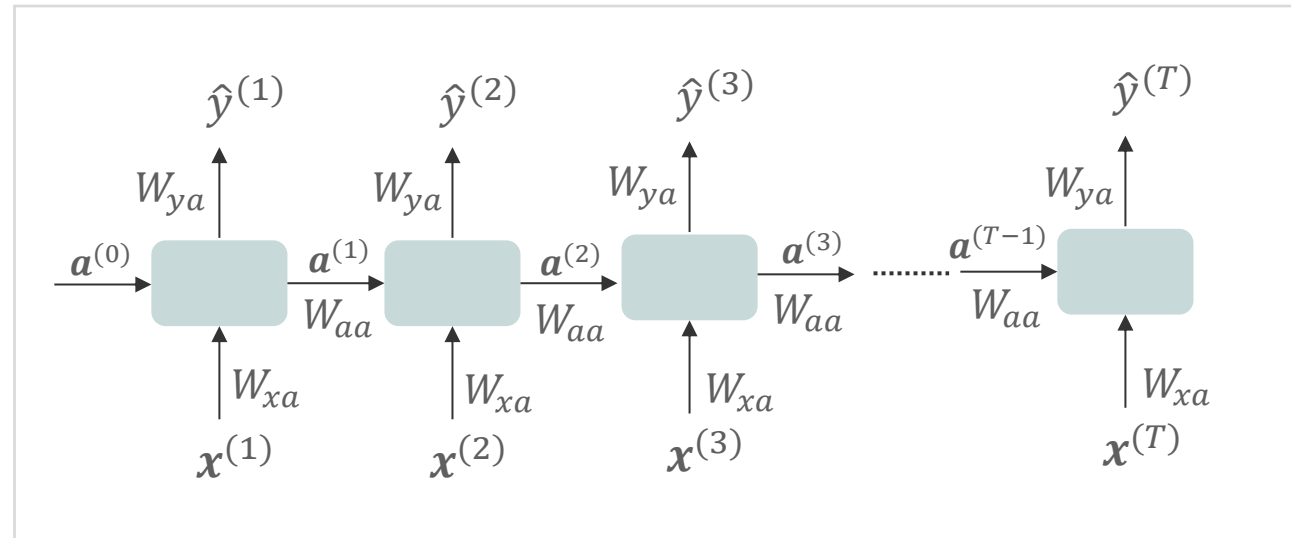
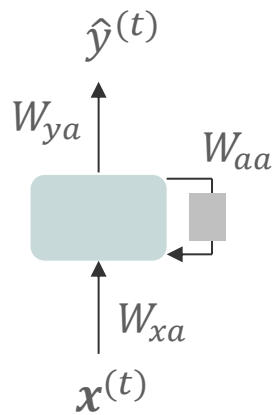
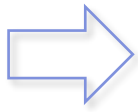
$$\mathbf{x} = (x_1, x_2, x_3)$$

Por ejemplo, 4
neuronas en la capa
recurrente



Se tienen pesos tanto para la
entrada como para las conexiones
recurrentes

$$\mathbf{a}^{(t)} = f(W_{aa}\mathbf{a}^{(t-1)} + W_{xa}\mathbf{x}^{(t)} + \mathbf{b})$$



Para calcular la salida de la red en el tiempo t :

$$a^{(0)} = 0$$

$$a^{(1)} = f_1(W_{aa}a^{(0)} + W_{xa}x^{(1)} + b_1) \quad f_1 \text{ en general } \tanh, \text{ReLU}$$

$$\hat{y}^{(1)} = f_2(W_{ya}a^{(1)} + b_2) \quad f_2 \text{ dependerá del problema.}$$

Por ejemplo, sigmoide

⋮

$$a^{(t)} = f_1(W_{aa}a^{(t-1)} + W_{xa}x^{(t)} + b_1)$$

$$\hat{y}^{(t)} = f_2(W_{ya}a^{(t)} + b_2)$$

- Se utilizan las **mismas matrices** de pesos en cada paso de tiempo. Esto asegura que se utilizará la misma transformación (la red aprende a identificar patrones en una secuencia, independiente del lugar en el que este patrón aparece).
- Entrenamiento: retropropagación de error a través del tiempo.
- Para cada peso, el algoritmo calcula el gradiente del error en cada paso de tiempo.
- Pero, al momento de la actualización, ¡solo hay un peso para actualizar!

RNN PASO A PASO

Las Redes Neuronales Recurrentes procesan secuencias paso a paso manteniendo una "memoria" del pasado.

IDEA CLAVE



En cada paso, la red recibe una entrada, actualiza su estado (memoria) y puede producir una salida.

1 ¿QUÉ ES UNA SECUENCIA?

Una secuencia es una lista ordenada de elementos.
Ejemplos: palabras en una oración, precios de acciones en el tiempo, notas de audio, etc.

Ejemplo: oración

"El gato está en la casa"

x_1

x_2

x_3

x_4

x_5

El gato está en la casa

Se procesa un elemento a la vez

2 EL PROBLEMA

Una red tradicional (como un MLP) no recuerda lo que vio antes. Pero para entender el contexto, necesitamos memoria.

Red tradicional (sin memoria)

$x_t \rightarrow \text{Red} \rightarrow y_t$

Necesitamos memoria

$x_t \rightarrow \text{Red} \rightarrow y_t$

La salida depende del pasado

3 IDEA CENTRAL: ESTADO OCULTO

La RNN mantiene un estado oculto (h_t) que resume la información de todo lo que ha visto hasta ahora. Ese estado se actualiza en cada paso.

Pasado

h_{t-1}

x_t

RNN

Nuevo estado

h_t

Salida (opcional)

y_t

4 CÓMO FUNCIONA (POR PASO)

En el tiempo t :

- 1 Recibe la entrada actual x_t
- 2 Usa el estado anterior h_{t-1}
- 3 Actualiza el estado h_t
- 4 (Opcional) Genera salida y_t

Operaciones internas (forma general)

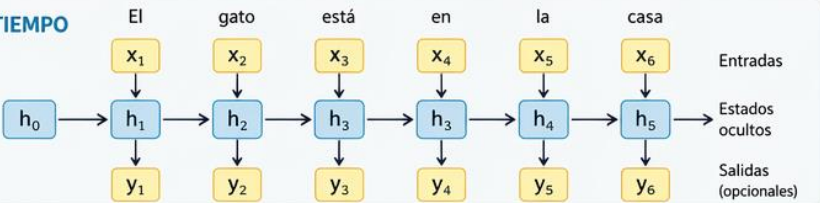
$$h_t = f(W_h h_{t-1} + W_x x_t + b_h)$$

$$y_t = g(W_y h_t + b_y)$$

h_t : estado oculto
 x_t : entrada en el tiempo t
 y_t : salida en el tiempo t
 W_h, W_x, W_y : pesos aprendibles
 f, g : funciones (ej. tanh, softmax)

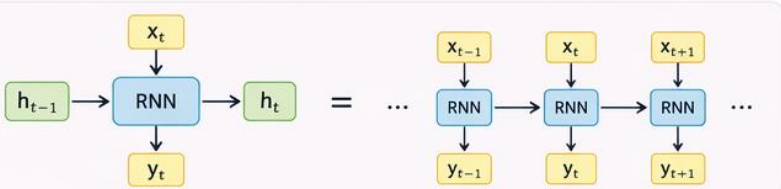
5 VISTA DESEENROLLADA EN EL TIEMPO

La misma red se repite en cada paso de tiempo, compartiendo los mismos pesos.



6 VISTA COMPACTA (RECURRENTE)

En lugar de desenrollar, podemos representar la RNN de forma compacta con un bucle.



7 ¿PARA QUÉ SIRVE?

- Las RNN se usan en muchos problemas donde el contexto importa:
- Procesamiento de lenguaje natural
 - Predicción de series temporales
 - Reconocimiento de voz
 - Análisis de video, música, etc.

Ejemplos de aplicaciones

 Traducción automática

 Predicción de ventas

 Reconocimiento de voz

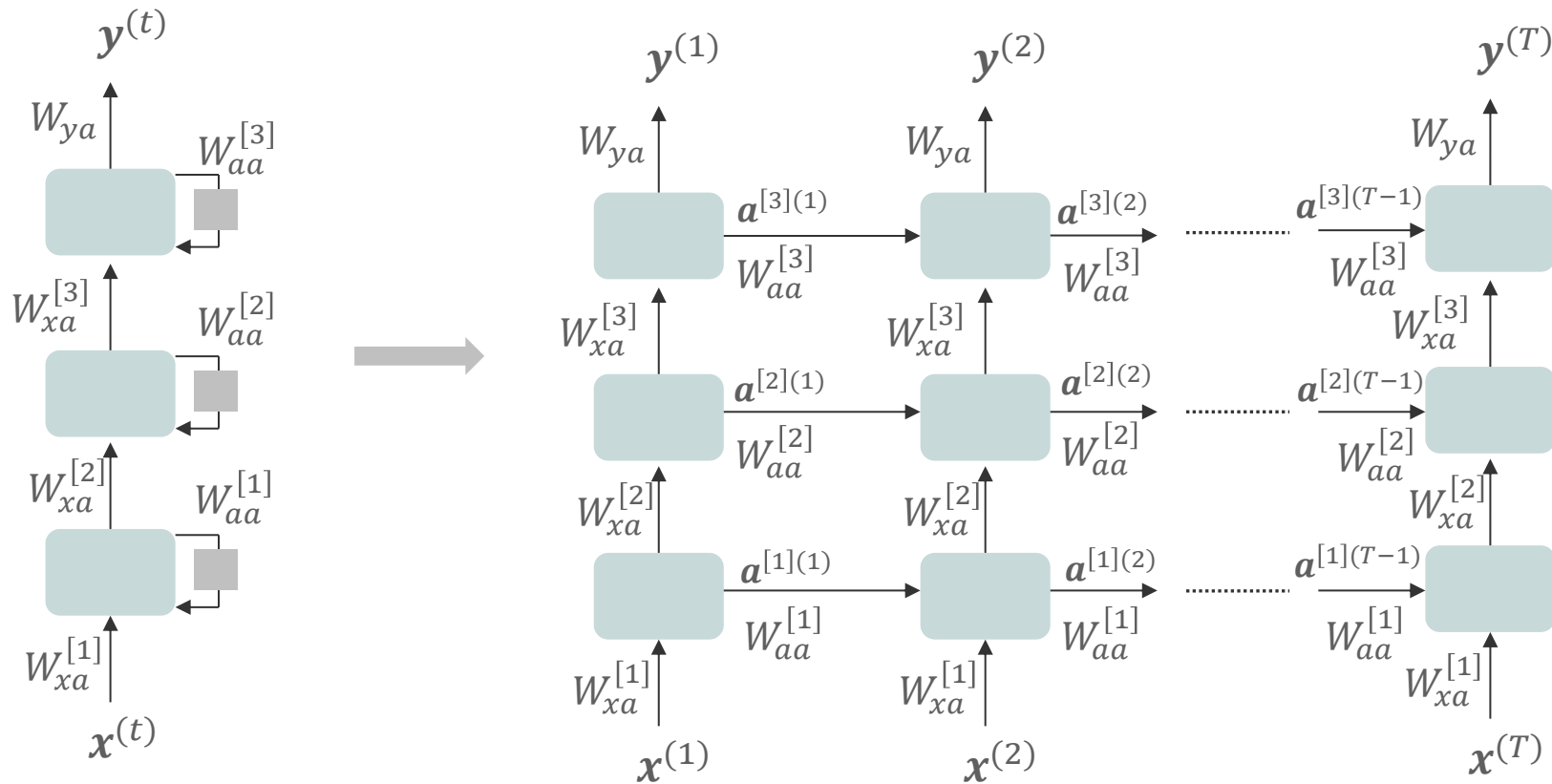
 Generación de texto o video

★ EN RESUMEN

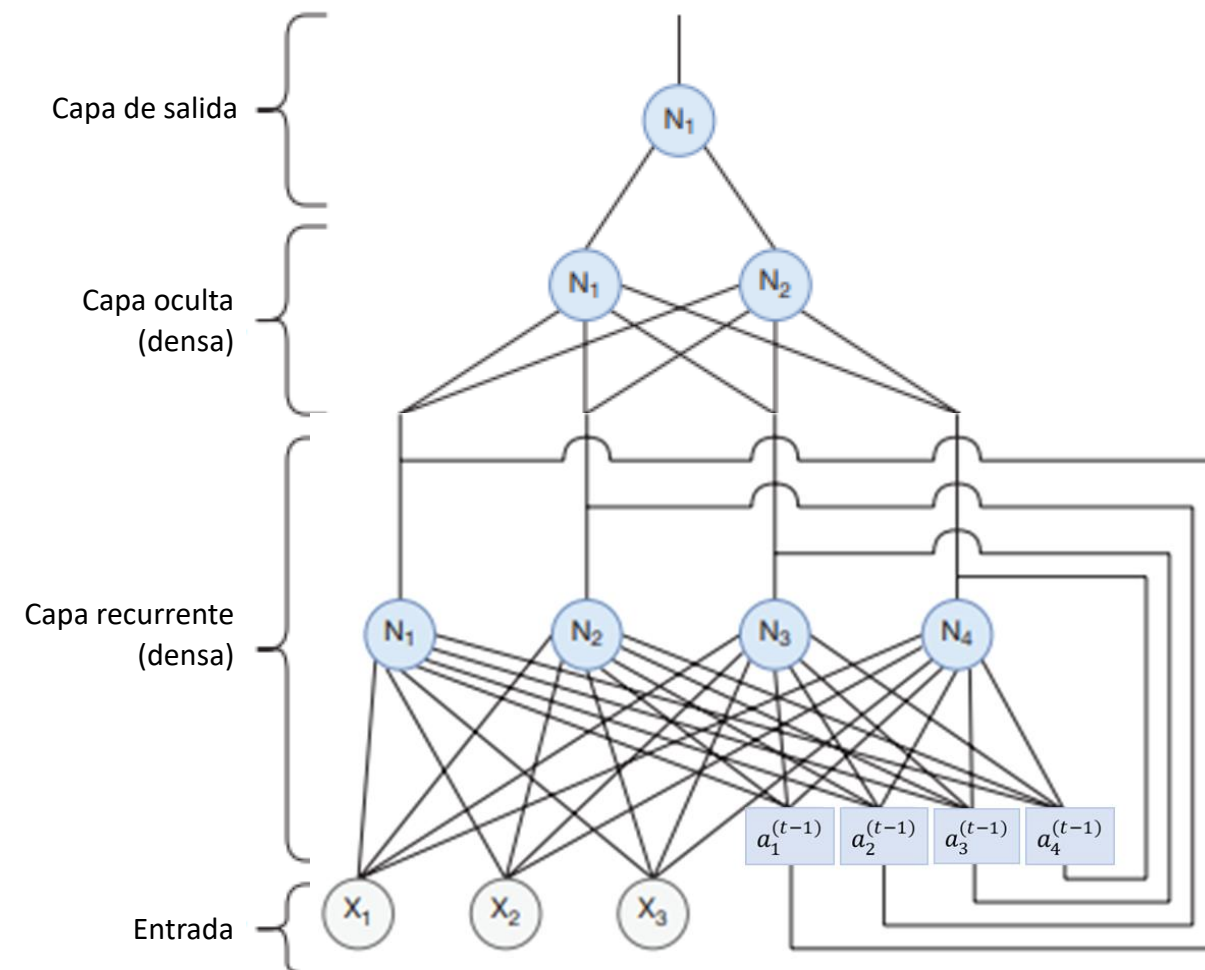
La RNN mantiene una memoria (estado oculto) que viaja en el tiempo. Eso le permite entender el contexto y tomar mejores decisiones en secuencias.



❑ Redes profundas recurrentes



Por ejemplo:
$$a^{[2]}(2) = f^{[2]}(W_{aa}^{[2]} a^{[2]}(1) + W_{xa}^{[2]} a^{[1]}(2) + b^{[2]})$$



Por ejemplo,

```
# Create RNN model
model = Sequential()
model.add(SimpleRNN(128, activation='relu',
                    input_shape=(None, 1)))
model.add(Dense(1, activation='linear'))
model.compile(loss='mean_squared_error', optimizer = 'adam',
              metrics = ['mean_absolute_error'])
```

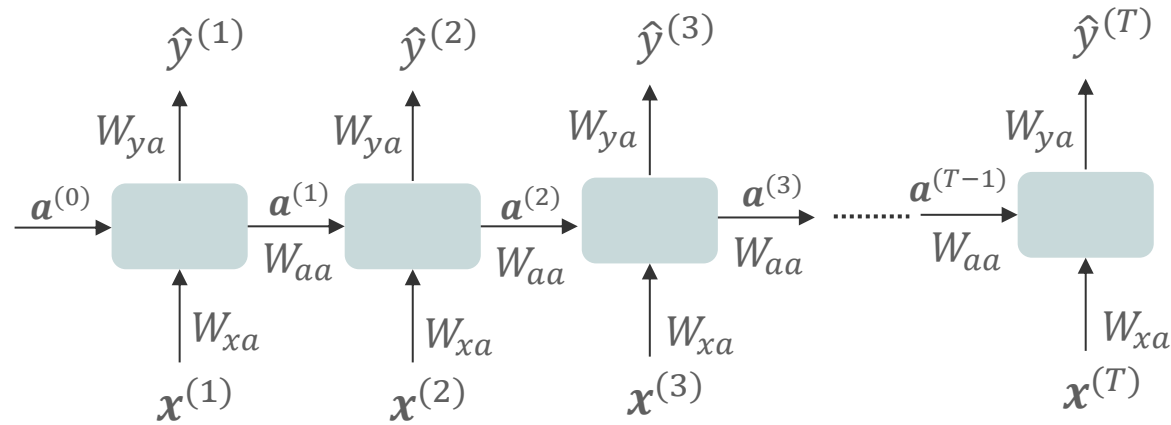
```
model.summary()
```

Layer (type)	Output Shape	Param #
=====		
simple_rnn_1 (SimpleRNN)	(None, 128)	16640

dense_1 (Dense)	(None, 1)	129
=====		
Total params: 16,769		
Trainable params: 16,769		
Non-trainable params: 0		

❑ Entrenamiento de RNN

- Algoritmo de retropropagación del error a través del tiempo (*backpropagation through time*).



- *Función de costo:*
$$J(\mathbf{W}) = \sum_{i=1}^T C(\hat{y}^{(t)}, y^{(t)})$$
 Donde C dependerá de la salida a predecir.

Por ejemplo:

$$C(\hat{y}^{(t)}, y^{(t)}) = -y^{(t)} \log(\hat{y}^{(t)}) - (1 - y^{(t)}) \log(1 - \hat{y}^{(t)})$$

Para aplicar BP se asume entonces que cada paso de tiempo utiliza matrices “diferentes” (tratar de capturar la contribución para cada entrada).

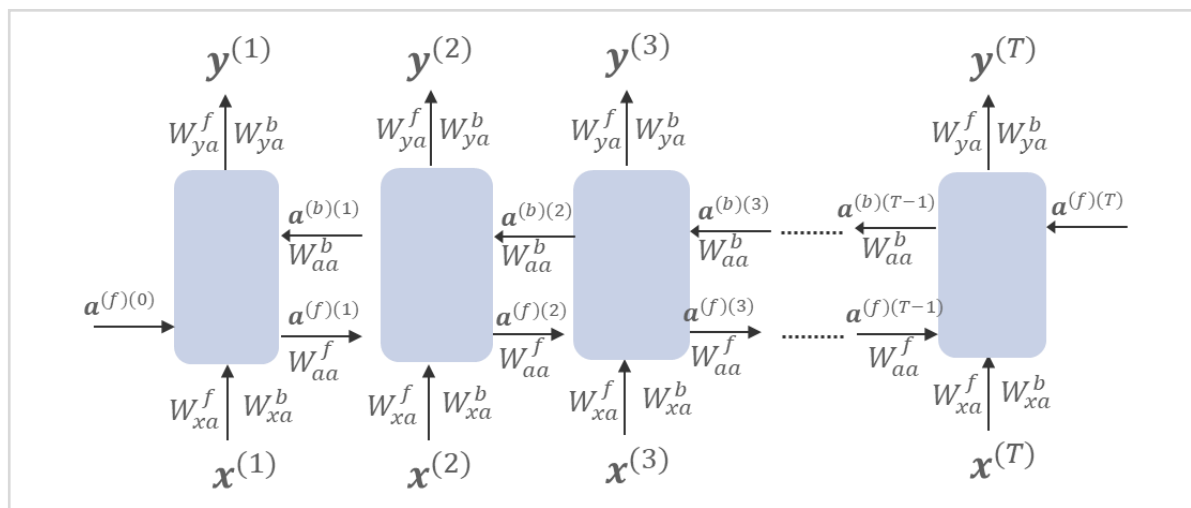
- **Paso hacia adelante** (*forward*): para cada entrada el algoritmo hace una predicción a través del tiempo (genera una salida).
- **Paso hacia atrás** (*back forward*): se calcula el error utilizando la función de costo. Se calcula el gradiente en cada paso de tiempo y se suma la contribución de cada uno:

$$\frac{\partial J(\mathbf{W})}{\partial w_{xa}^{(t)}}, \quad \frac{\partial J(\mathbf{W})}{\partial w_{aa}^{(t)}}, \quad \frac{\partial J(\mathbf{W})}{\partial w_{ya}^{(t)}} \quad t = 1 \dots T \quad \left\{ \begin{array}{l} \frac{\partial J(\mathbf{W})}{\partial w_{xa}} = \sum_{i=1}^T \frac{\partial J(\mathbf{W})}{\partial w_{xa}^{(i)}} \\ \frac{\partial J(\mathbf{W})}{\partial w_{aa}} = \sum_{i=1}^T \frac{\partial J(\mathbf{W})}{\partial w_{aa}^{(i)}} \\ \frac{\partial J(\mathbf{W})}{\partial w_{ya}} = \sum_{i=1}^T \frac{\partial J(\mathbf{W})}{\partial w_{ya}^{(i)}} \end{array} \right.$$

- **Paso de actualización**: se ajustan los parámetros según la regla de descenso por el gradiente.

Redes bidireccionales (BRNN)

- Para algunos problemas se pueden mejorar los resultados si se tiene conocimiento, no solo de las entradas pasadas, sino también de los estados futuros. Por ejemplo, reconocimiento de textos manuscritos, secuencias biológicas, reconocimiento de voz,...
- Esto implica un mayor conocimiento del contexto. ¿Cómo incluir estos estados futuros en la generación de la salida de la red?



$$a^{(f)}(t) = f_1(W_{aa}^f a^{(t-1)} + W_{xa}^f x^{(t)} + b_1^f)$$

$$a^{(b)}(t) = f_1(W_{aa}^b a^{(t+1)} + W_{xa}^b x^{(t)} + b_1^b)$$

$$y^{(t)} = f_2(W_{ya}^f a^{(f)}(t) + W_{ya}^b a^{(b)}(t) + b_2)$$

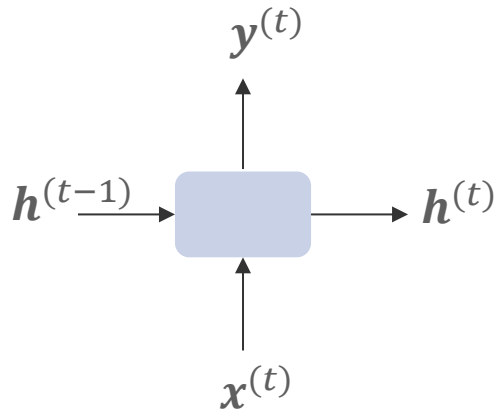
¿Cómo sería el entrenamiento?

- Paso hacia adelante (forward):** se calculan los valores de activación de las capas ocultas en cada dirección (de manera independiente). Luego se determina la salida de la red a partir de estos estados de activación.
- Paso hacia atrás (back forward):** se calcula el error utilizando la función de costo. Se calcula el gradiente en cada paso de tiempo, hacia atrás y hacia adelante de manera independiente, y se suma la contribución de cada uno.
- Paso de actualización:** se ajustan los parámetros según la regla de descenso por el gradiente.

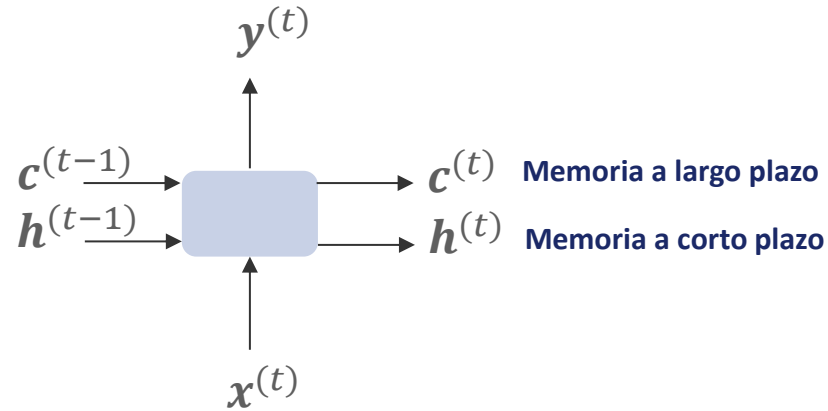
❑ Redes con memoria a corto y largo plazo

- Un problema que se presenta cuando se procesan secuencias muy largas: desvanecimiento de la memoria (en los primeros pasos de tiempo).
- Se pierden dependencias temporales de largo plazo (“olvido” gradual de las primeras entradas).
- Solución: utilizar celdas de memoria de largo plazo.
- Por ejemplo: **LSTM (Long Short-Term Memory)**, Gated Recurrent unit (GRU),

En una RNN básica:



Si se utiliza una celda LSTM:



- Diseñadas para procesar de manera explícita las dependencias a largo plazo.
- Son capaces de “recordar”, pero también de “olvidar”.
- Mantiene dos estados: el de corto plazo ($h^{(t)}$) y el de largo plazo ($c^{(t)}$).

Son redes recurrentes con memoria controlada mediante “puertas” que permiten aprender qué información mantener o descartar a lo largo del tiempo, resolviendo el problema de olvidar dependencias importantes.

¿Cómo funcionan?

LSTM

Redes Neuronales con Memoria Inteligente

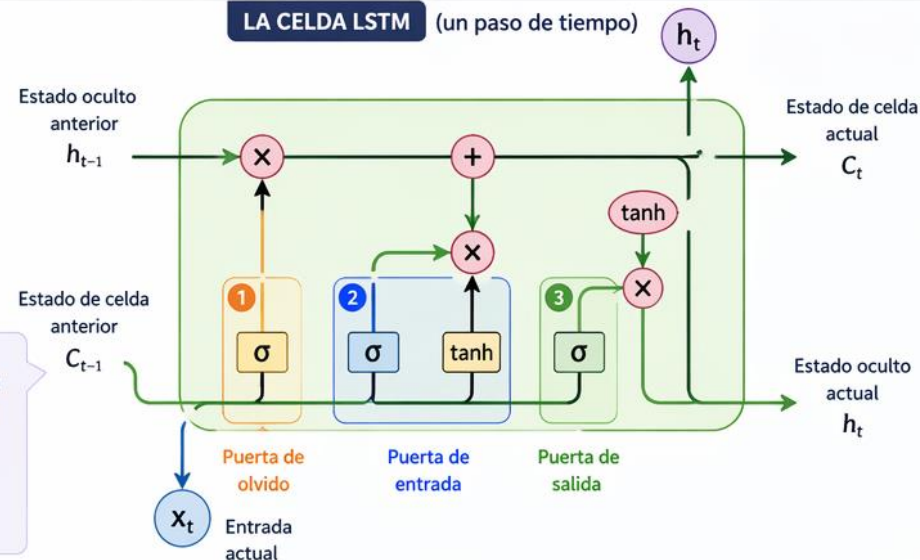
¿POR QUÉ LSTM?

Las RNN tradicionales tienen dificultad para recordar información importante durante mucho tiempo.



LSTM soluciona esto usando una memoria interna y tres "puertas" que controlan qué recordar, qué olvidar y qué usar.

LA CELDA LSTM (un paso de tiempo)



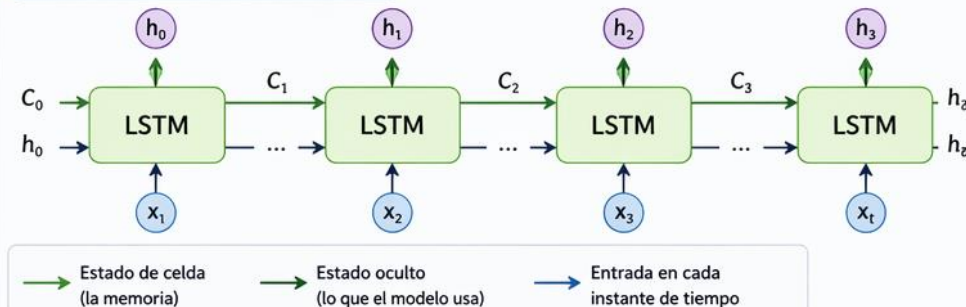
σ = Sigmoide (0 a 1)

$\times$ = Multiplicación elemento a elemento

$+$ = Suma elemento a elemento

$\tanh$ = Tangente hiperbólica (-1 a 1)

LSTM DESENNROLLADA EN EL TIEMPO



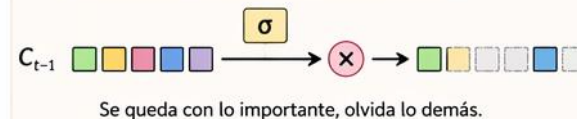
★ EN RESUMEN

LSTM tiene una memoria interna (estado de celda) y tres puertas que le permiten controlar qué información olvidar, qué información guardar y qué información usar en cada momento.

LAS TRES PUERTAS

1 PUERTA DE OLVIDO

Decide qué información del estado de celda anterior ya no es relevante y debe eliminarse.



Se queda con lo importante, olvida lo demás.

2 PUERTA DE ENTRADA

Decide qué información nueva se debe guardar en el estado de celda.



Actualiza la memoria con nueva información útil.

3 PUERTA DE SALIDA

Decide qué parte del estado de celda actual se usará como salida (estado oculto).



Usa solo lo necesario para generar la salida.

UNA ANALOGÍA FÁCIL

1. OLVIDA

Tacha lo que ya no importa.



2. GUARDA

Escribe lo nuevo que es importante.



3. USA

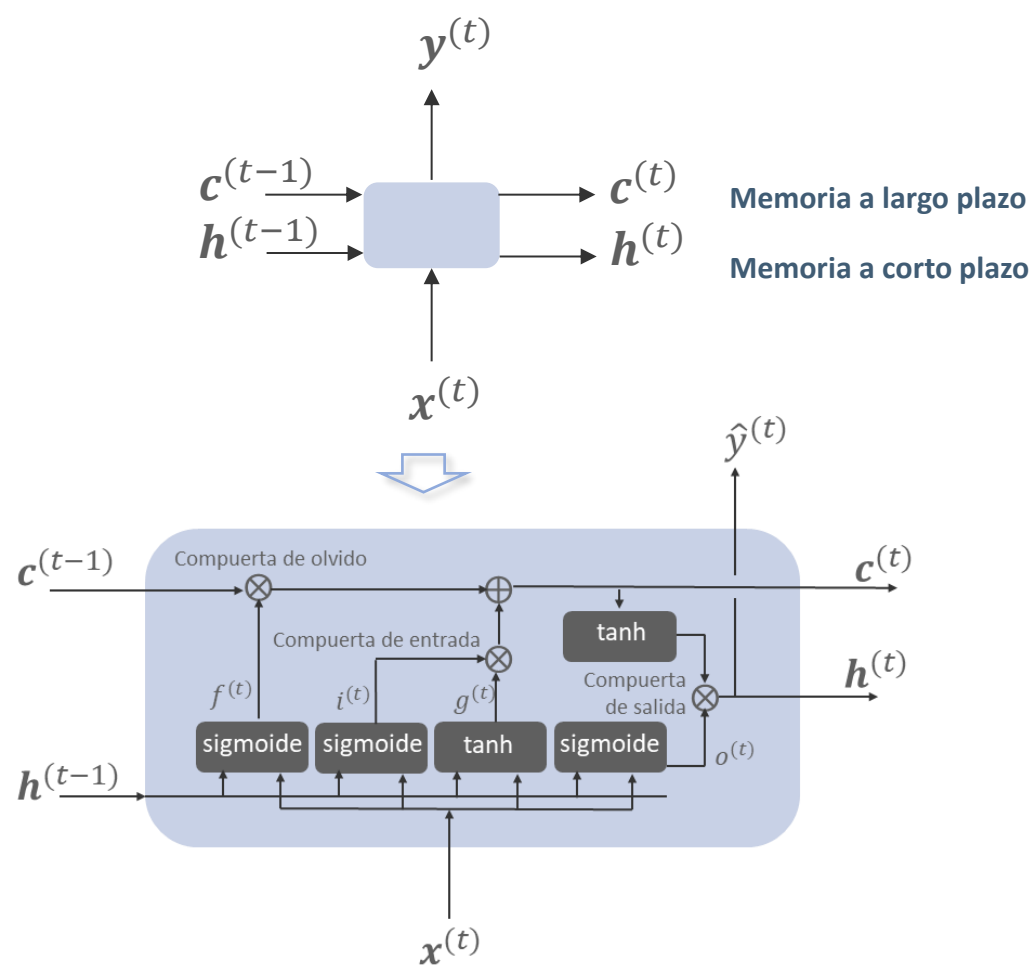
Usa solo lo necesario para responder ahora.



Imagina que la LSTM es como una persona que toma notas:



Esto le permite entender dependencias largas y recordar información importante por más tiempo.



- Actualización de la memoria a corto plazo:

$$c^{(t)} = c^{(t-1)} \odot f^{(t)} + i^{(t)} \odot g^{(t)}$$

- Actualización de la memoria a largo plazo:

$$h^{(t)} = o^{(t)} \odot \tanh(c^{(t)})$$

- Salida: $y^{(t)} = f_y(W_{ya}a^{(t)} + b_y)$

Además de la celda de estado (c) se introducen tres compuertas (*gates*). Como la función de activación que utilizan estas capas es la sigmoide (con salida entre 0 y 1), ellas pueden cerrar o abrir las compuertas mediante la operación de multiplicación (0, la cierra; 1 la abre):

- La compuerta de olvido (controlada por $f^{(t)}$) determina qué partes del estado a largo plazo deben borrarse.

$$f^{(t)} = \theta(W_{af}h^{(t-1)} + W_{xf}x^{(t)} + b_f)$$

- La compuerta de entrada (controlada por $i^{(t)}$) controla cuánta información debe agregarse al estado a largo plazo.

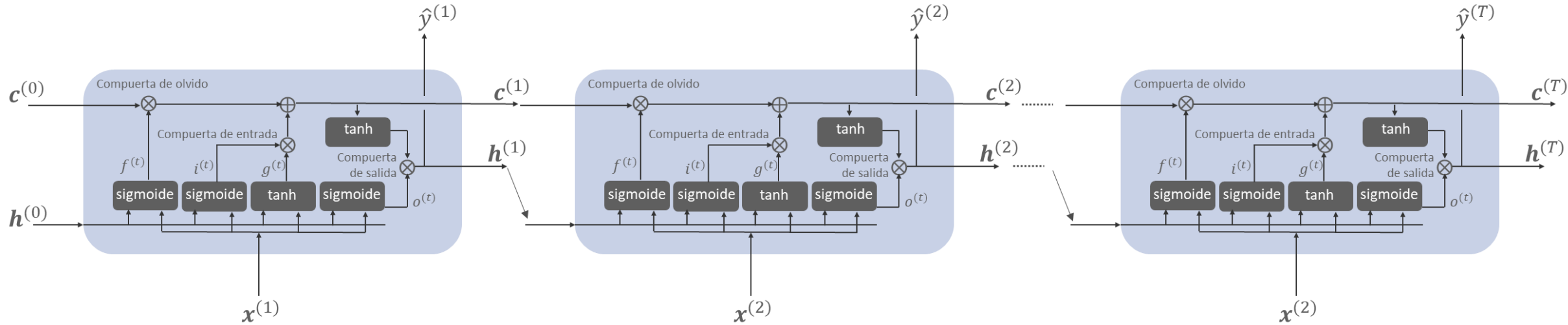
$$i^{(t)} = \theta(W_{ai}h^{(t-1)} + W_{xi}x^{(t)} + b_i)$$

- La compuerta de salida (controlada por $o^{(t)}$) determina qué partes del estado a largo plazo deben enviarse al siguiente paso de tiempo a través de $h^{(t)}$ y a la salida $y^{(t)}$.

$$o^{(t)} = \theta(W_{ao}h^{(t-1)} + W_{xo}x^{(t)} + b_o)$$

- A través de $g^{(t)}$ se procesa la entrada $x^{(t)}$ y el estado anterior $h^{(t-1)}$ (memoria a corto plazo). La salida de esta capa se almacena parcialmente en la memoria a largo plazo.

$$g^{(t)} = \tanh(W_{ag}a^{(t-1)} + W_{xg}x^{(t)} + b_g)$$



Por ejemplo,

Se aplica en las conexiones entre capas (entrada → recurrente, recurrente → salida).

```
model = Sequential()
model.add(LSTM(128, return_sequences=True,
               dropout=0.2, recurrent_dropout=0.2,
               input_shape=(None, encoding_width)))
model.add(LSTM(128, dropout=0.2,
               recurrent_dropout=0.2))
model.add(Dense(encoding_width, activation='softmax'))
model.compile(loss='categorical_crossentropy',
              optimizer='adam')
```

Para producir una salida en cada paso de tiempo

Se aplica en las conexiones en las conexiones de la memoria temporal de un paso al siguiente. Sirve para regularizar la dependencia temporal.

Ejemplo: se quiere desarrollar un sistema automatizado que clasifique reportes internos de una empresa según el tipo de incidente registrado. Cada dato está compuesto por descripciones textuales de los hechos, antecedentes del área involucrada y un incidente principal, el cual se asocia a una de 10 clases posibles, como Fallas Técnicas, Problemas Logísticos, Incumplimientos de Proveedores, Riesgos de Seguridad, entre otras. Como primera aproximación a la solución, se propuso construir un modelo de clasificación utilizando una red neuronal recurrente con dos capas ocultas: la primera con 128 neuronas y la segunda con 32. La capa de salida utiliza la función de activación softmax. El vocabulario corporativo considerado en este contexto consta de 18200 palabras.

Si se utiliza un embedding de 512 ¿Cuál es el tamaño de cada una de las matrices de pesos de esta red neuronal? Incluya la capa de embedding.

- Mecanismos de atención
- La arquitectura Transformer
- Modelos de lenguaje